UNIVERSITÀ DEGLI STUDI DI PERUGIA

Dipartimento di Matematica e Informatica

Progetto ReBin

Autori:

Tommaso Rossi
Giordano Ragnacci
Samuele Grenga

A.A. 2025/2026

Indice

1 Descrizione del progetto **2**

2 Descrizione dei requisiti **2**

3 Struttura del programma **2**

4 Run & replay **2**

4.1 Run 2

4.2 Replay 3

4.3 gdb 4

4.4 Utilizzo di stdbuf 4

5 Logger **4**

1 Descrizione del progetto

Il progetto si chiama "ReBin", permette di poter interagire con file binari, registrare le sessioni di input/output, e poter fare replay delle azioni effettuate in una sessione così da ottenere lo stesso risultato.

Repository GitHub: `https://github.com/ragn0/rebin`

2 Descrizione dei requisiti

- **Python 3**
- **pwntools**: libreria su cui si basa il progetto, utilizzata per eseguire il binario e interfacciarsi con esso
- **stdbuf** (Opzionale, ma consigliata): utility di sistema che ha lo scopo di disabilitare il buffering dei file binari

3 Struttura del programma

- **harness.py**: funge da main del programma. Controllando gli argomenti passati da riga di comando quando si esegue il file smista l'esecuzione verso registrazione o replay.
- **logger.py**: si occupa della struttura e del salvataggio dei dati. Crea il file JSON salvando i dati della sessione.
- **replay.py**: permette il replay di una sessione passata. Legge i dati dal file JSON riavvia il binario e invia gli input registrati.
- **run.py**: permette l'esecuzione del binario. Avvia il processo e salva l'input e l'output passandoli al logger.
- **utilities.py**: modulo di supporto che contiene funzioni utili a tutti i programmi così da evitare ridondanza. Come la risoluzione dei percorsi dei file e la scrittura atomica del JSON.

4 Run & replay

4.1 Run

Il programma avvia il file binario per permettere all'utente d'interagirvi, il sistema intercetterà e salverà il traffico. Le sue fasi principali sono:

- **Preparazione dell'ambiente**: viene istanziato il logger che si occuperà di registrare la sessione e viene risolto il percorso del binario.
- **Esecuzione del processo**: per l'esecuzione del processo, utilizziamo uno pseudo terminale (PTY) generando una coppia master/slave, allo slave collegheremo il binario che verrà avviato utilizzando `process()` di pwntools, poi tramite il master leggeremo e scriveremo (Le prime due fasi sono condivise da tutte e tre le modalità).
  - *Perché usiamo uno pseudo-terminale?* Non utilizziamo solo process per eseguire il binario, perché alcuni programmi controllano verso dove viene inviato l'output: se è una pipe (default di process) utilizzano molto spesso un buffering, che attende di aver raggiunto una certa quantità di byte prima d'inviare. Invece se è un terminale o usa un buffering per riga, cioè fino ad una new line (\n) o non lo utilizza proprio.

- *Perché usiamo termios?* La libreria termios, risolve il problema dell'eco. Il terminale che creiamo con PTY, ha di default la funzionalità ECHO abilitata ossia che "rimbalza" quello che stai scrivendo per mostrartelo. A noi è d'intralcio questa funzionalità perché già gestiamo l'input, infatti l'utente scriverà sul terminale "principale" e verrà mandato allo pseudo utilizzando il master, non c'interessa che ci mostri cosa abbiamo scritto perché sennò il nostro programma lo considererebbe un output del binario. Fortunatamente e' possibile disattivare questa funzionalita settando a 0 il bit della flag `termios.ECHO` tramite un & tra gli attributi di default di termios e la flag `termios.ECHO` annullata tramite il NOT bitwise.

- **Interazione**: una volta eseguito il binario, si entra in un loop in cui avviene l'interazione dell'utente. Utilizzando la funzione `select.select()` controlliamo quale tra i due flussi è pronto, se l'output del binario o l'input dell'utente. Quando l'utente digita qualcosa il dato viene inviato al master e registrato dal logger, stesso discorso per l'output che però verrà anche stampato a video.

  - *Perché select.select()?* Di norma le operazioni di lettura sono bloccanti, perciò il programma di ferma ad aspettare un mio input non potendo accorgersi se nel frattempo è stato stampato qualcosa dal binario. Select risolve questo stallo, perché si attiva solo quando uno dei due flussi è pronto, cioè ci sono byte da leggere. Ritorna una lista (rlist) che contiene i flussi pronti, si fa un if poi per la gestione.

- **Chiusura**: la sessione termina o tramite `Ctrl+D` da parte dell'utente o se il binario va in crash. Si recupera il codice di uscita e viene salvato dal logger che chiuderà il salvataggio della sessione.

4.2 Replay

Il programma avvia il processo e simula l'utente leggendo gli input salvati nel file relativo alla sessione precedente. Le sue fasi principali sono:

- **Lettura e validazione**: viene aperto il file JSON passato come argomento e viene estratto il nome del file binario.

- **Esecuzione del processo**: come in run.py istanziamo uno pseudo-terminale che colleghiamo all'esecuzione del binario.

  - *Perché termios non viene utilizzato?* Termios non viene utilizzato perché ora non dobbiamo scrivere su nessun file di log. Poi il fatto che vengano stampati a video è comodo per vedere i comandi che il programma sta iniettando.

- **Replay**: il programma legge gli input salvati nel file JSON e per ognuno, lo invia al programma utilizzando il master per poi raccogliere l'output generato.

  - *Utilizzo di select.select():* a differenza di run.py, qua select non è in ascolto di due canali (tastiera e output del binario), infatti è in ascolto solo dell'output del binario e per capire la fine delle risposte, si utilizza un timeout (0.1s) dopo il quale s'invia altro input dal file.

- **Terminazione**: quando gli input salvati nel file sono terminati, attende la terminazione naturale del processo stampando poi a video il codice di uscita.

4.3 gdb

Abbiamo anche aggiunto la possibilità di wrappare l'esecuzione del binario con l'utilizzo di gdb, cosi' facendo l'utente non sarà limitato alla sola esecuzione del binario, ma anche al debugging e all'utilizzo delle funzioni fornite da gdb. Si noti che vengono salvati sia gli input inviati dall'utente, che qualsiasi output mandato sia da gdb che dal binario esaminato. Le fasi principali sono praticamente le stesse della modalita run, con la differenza che, anziche' usare come cmd soltanto il binario o stdbuf + il binario, andiamo ad avviare il binario tramite gdb.

4.4 Utilizzo di stdbuf

Nonostante l'utilizzo dello pseudo-terminale per evitare il buffering a blocchi, potrebbe accadere che anche il buffering per riga ci dia problemi. Per prevenire questa situazione sia per run.py che per replay.py utilizziamo stdbuf (se presente), che al comando di avvio del processo applicherà dei flag (-i0 -o0 -e0) che forzeranno l'stdin, stdout e l'stderr ad operare in modalità non bufferizzata. La conseguenza sarà che ogni byte verrà subito reso disponibile sul canale master del nostro PTY. Se stdbuf non dovesse essere disponibile, semplicemente si va avanti accettando il rischio che eventuali prompt senza terminatore di riga (\n), potrebbero restare bufferizzati.

5 Logger

Il programma raccoglie in modo strutturato le interazioni così da permettere poi la riproduzione. Il salvataggio è delegato alla classe SessionLogger, le cui caratteristiche principali sono:

- **Inizializzazione**: al momento dell'istanza il logger crea uno scheletro dati che contiene:
  - ID basato su data e timestamp
  - Percorso del file binario
  - Metadati di sistema (versione python, ...)
  - Riferimento temporale iniziale
- **Registrazione**: i metodi `record_input` e `record_output`, vengono richiamati nel ciclo d'interazione di run.py. Insieme ai byte prodotti, salva anche il tempo trascorso dall'avvio del processo facendo la sottrazione con il riferimento temporale iniziale.
- **Gestione della chiusura**: il metodo `finalize()`, viene invocato al termine del processo in run.py. Il logger cattura l'esito dell'esecuzione (codice di uscita ed eventuale messaggio) salvandoli.
- **Scrittura atomica**: per prevenire la corruzione dei dati in caso che il programma terminasse inaspettatamente, il metodo `save()` delega il salvataggio in memoria alla funzione `atomic_write_json`. Questa funzione scrive l'intero log in un file temporaneo, solo a scrittura completa salverà il file definitivo sulla cartella di destinazione.

Questo sotto è come appare lo scheletro della classe `SessionLogger`:

```
class SessionLogger:
    def __init__(
        self,
        binary_path: Optional[str],
        env: Optional[Dict[str, str]] = None,
        metadata: Optional[Dict[str, Any]] = None,
    ) -> None:
        self.t0 = monotonic_s()
        self.log: Dict[str, Any] = {
            "id": time.strftime("%Y%m%d_%H%M%S"),
```

```python
            "target": binary_path,
            "timestamp": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
            "args": [],
            "env": env or {},
            "inputs": [],
            "outputs": [],
            "exit_code": None,
            "error": None,
            "metadata": {
                "python_version": platform.python_version(),
                "platform": platform.platform(),
            } | (metadata or {}),
        }
```

Questo è un esempio di un file di log completo, relativo ad un'esecuzione:


```json
{
  "id": "20260218_174053",
  "target": "binaries/ctf",
  "timestamp": "2026-02-18T16:40:53Z",
  "args": [],
  "env": {},
  "inputs": [
    {
      "t": 1.3406503800006249,
      "data": "ciao\n"
    },
    {
      "t": 2.5376686790004896,
      "data": "ciao\n"
    },
    {
      "t": 8.854514325999844,
      "data": "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\
    nAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\
    nAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\n"
    }
  ],
  "outputs": [
    {
      "t": 0.004446839000593172,
      "data": "Ciao\r\n"
    },
    {
      "t": 1.3406793000003745,
      "data": "I guess you're not cool enough to see my secret\r\n"
    },
    {
      "t": 2.5377043290000074,
      "data": "I guess you're not cool enough to see my secret\r\n"
    },
    {
      "t": 8.854569475000062,
      "data": "Wow! Here's a secret.\r\n"
    }
  ],
  "exit_code": -11,
  "error": null,
  "metadata": {
    "python_version": "3.14.2",
    "platform": "Linux-6.18.9-200.fc43.x86_64-x86_64-with-glibc2.42"
  }
}
```